

KAVACH

Testing and Benchmarking Report

1. Executive Summary

This report records how KAVACH was tested and what it achieved. Every number below comes from an artefact that can be regenerated from the repository: the automated test log, the benchmark JSON produced by the served model, and command-line runs captured on 29 September 2026. The model being evaluated is identified by its SHA-256 hash, so the results are tied to one exact set of weights.

110 / 110	0.86 vs 0.49	78%	0.96–0.97
automated tests passed	F1 vs required baseline	fewer false alarms	forecast AUC, 1–3 steps

Headline result	Logistic regression	KAVACH (NetJEPA)	Change
F1 score (detection)	0.49	0.86	**+76%**
ROC AUC (detection)	0.73	0.96	**+31%**
False-positive rate	29.3% (1,568 false alarms)	6.3% (339 false alarms)	**−78% false alarms**
Forecast AUC, t+1 / t+2 / t+3	0.74 / 0.74 / 0.73	0.97 / 0.97 / 0.96	**+0.23**
Attack-onset forecast AUC	n/a	0.90–0.93	**new capability**
90% prediction-interval coverage	n/a	95%	**well calibrated**
Flow + packet vs flow only (F1)	—	0.87 vs 0.62	**+40%**
Packet parser throughput vs Scapy	3,608 pkt/s	210,492 pkt/s	**58× faster**

> **Verification statement**
>
> All 110 automated tests passed in the run recorded in Appendix C. Benchmark figures were computed on **8,043 held-out windows** that the model never saw during training (the last 20% of every CIC-IDS-2017 day), with the decision threshold chosen on a separate calibration split. Known weaknesses are reported in Sections 8 and 11 alongside the strengths.

2. Test Environment and Methodology

2.1 System under test

Item	Value
Model	NetJEPA v2: heterogeneous graph transformer (2 layers) → GRU over 3 windows → Gaussian predictor
Weights file	data/processed/netjepa_weights.pt
Weights SHA-256	baa45b2008d6bd6a7f2a00286bd80ab6fe2ba9c3265a13c52eeb724f9c9e2a42
Parameters	1,279,231 (about 5 MB)
Feature schema	v2: 29 features per 5-second window (16 flow, 11 packet, 2 modality flags)
Context / horizon	3 windows of history; forecasts 3 windows (15 s) ahead, 64 Monte-Carlo samples
Calibration	Temperature 0.751; decision threshold 0.57 (chosen on a separate calibration split)
Hardware	CPU only, 2 vCPU; no GPU used for training or inference
Software	Python 3.11, PyTorch, PyTorch Geometric, scikit-learn, FastAPI, pytest 9.1

2.2 Data and split

- **Dataset:** CIC-IDS-2017 (Canadian Institute for Cybersecurity), both the labelled flow CSVs and the raw PCAP captures.
- **Split:** time-based, per day. The first 80% of each day trains the model; the last 20% is held out. This prevents the model from seeing the future of the same attack during training.
- **Sizes:** 32,103 training windows and 8,043 validation windows, of which 2,683 contain attack traffic.
- **Baseline:** logistic regression on the same 29 features, as required by the problem statement. A second baseline gives logistic regression the same three-window history NetJEPA receives.

2.3 Metrics

- **F1, precision, recall, false-positive rate** at the calibrated threshold, plus **ROC AUC** (threshold-free).
- **Forecast AUC:** does the predicted risk for window t+k match whether window t+k actually contains an attack?
- **Onset AUC:** the same, restricted to windows that are benign now. This is the true early-warning case.
- **Expected calibration error (ECE)** and **interval coverage** for the reliability of confidence scores.

3. Automated Test Suite

The repository contains 110 automated tests covering every stage of the pipeline, from packet parsing to the blockchain ledger and the tiered login. The complete suite was run on 29 September 2026 with `pytest -v`.

```
======================== 110 passed, 1 warning in 41.40s ========================
```

Test module	Tests	What it verifies
test_ingestion.py	8 / 8	Flow-CSV loading, PCAP parsing, retransmission and port-scan detection, batching
test_dataset_formats.py	4 / 4	Format auto-detection for CIC-IDS-2017, CSE-CIC-IDS-2018 and CTU-13
test_extract.py	9 / 9	5-second windowing, flow + packet aggregation, normalisation, merging both levels
test_state_builder.py	5 / 5	Graph construction: hosts as nodes, flows as edges, fixed-width state vectors
test_netjepa.py	8 / 8	NetJEPA forward pass, K-step rollout, EMA target update, embedding collapse checks
test_vicreg.py	2 / 2	VICReg loss runs and penalises representation collapse
test_v2_components.py	8 / 8	Fast parser vs Scapy field parity, supervised transitions, Gaussian predictor, offline CLI
test_train.py	3 / 3	Class-imbalance weighting in training
test_scoring.py	7 / 7	Infiltration head, forecast curves, MITRE stage classifier output validity
test_scoring_extras.py	3 / 3	Temperature calibration, remediation playbooks, concept-bottleneck overrides
test_shap_explainer.py	4 / 4	Feature attribution returns real feature names and respects limits
test_benchmark.py	4 / 4	Benchmark harness: baseline, checkpoint evaluation and caching
test_pipeline.py	8 / 8	End-to-end pipeline streaming from real CSV and PCAP through every model stage
test_results_store.py	7 / 7	Prediction store: save, query, alert thresholds, summaries
test_adapters.py	9 / 9	API adapters: flows, alerts, kill-chain state, risk score
test_api_extended.py	4 / 4	REST API: benchmark, feedback chaining, timeline, remediation, tamper demo
test_blockchain_ledger.py	8 / 8	Hash chain, tamper and deletion detection, Merkle batching, node sync, model updates
test_contracts_and_audit.py	3 / 3	3-node consensus contract, escalation contract, feedback-poisoning audit
test_auth_tiers.py	6 / 6	Three-tier login (password, TOTP, hardware key), lockout, per-tier response rules, two-person approval

```
============================= test session starts ==============================
platform linux -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
rootdir: /home/claude/kv2
plugins: anyio-4.13.0
...
tests/test_v2_components.py::test_batched_encoder_matches_single_path_and_predictor_is_gaussian PASSED [ 95%]
tests/test_v2_components.py::test_examples_carry_future_labels_for_transition_supervision PASSED [ 96%]
tests/test_v2_components.py::test_cli_predict_runs_offline PASSED        [ 97%]
tests/test_v2_components.py::test_novelty_detector_percentiles PASSED    [ 98%]
tests/test_vicreg.py::test_vicreg_loss_runs PASSED                       [ 99%]
tests/test_vicreg.py::test_vicreg_penalizes_collapse PASSED              [100%]
...
====================== 110 passed, 1 warning in 41.40s =======================
```

Figure 3.1 Proof of execution: start and end of the pytest run (full list in Appendix C)

4. Detection Benchmark

Binary detection: is this 5-second window part of an attack? All three models are scored on the same 8,043 held-out windows.

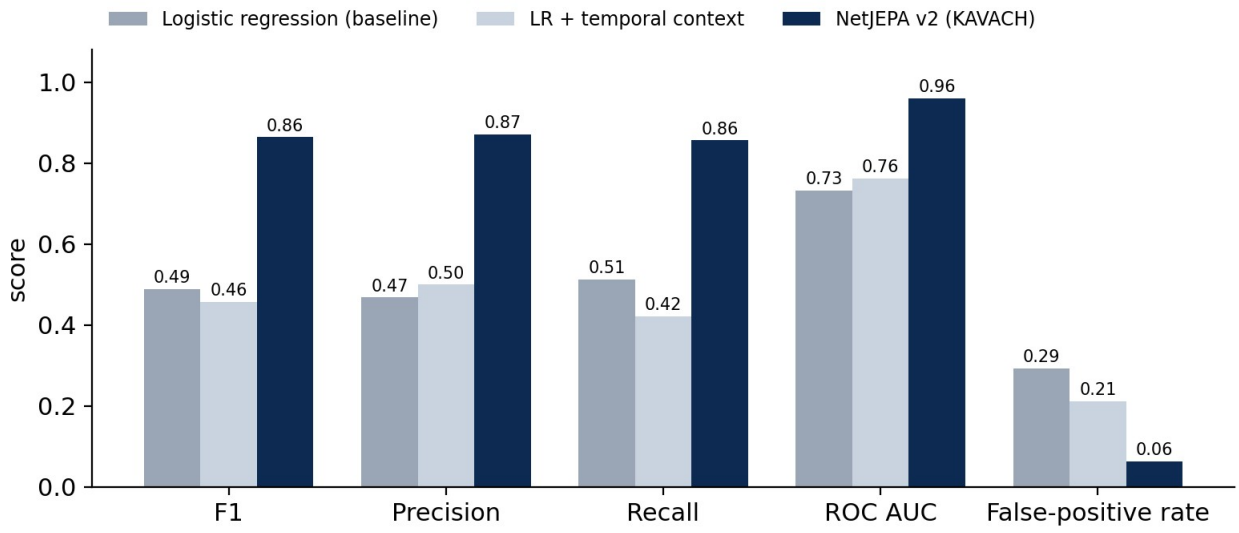


Figure 4.1 Detection metrics on held-out windows

Model	F1	Precision	Recall	FPR	AUC
Logistic regression (baseline)	0.489	0.468	0.513	29.3%	0.732
Logistic regression + 3-window context	0.457	0.500	0.421	21.1%	0.762
NetJEPA v2 (KAVACH)	**0.864**	**0.871**	**0.856**	**6.3%**	**0.960**

Confusion counts	True positives	False positives	True negatives	False negatives
Logistic regression	1,377	1,568	3,792	1,306
NetJEPA v2	**2,297**	**339**	**5,021**	**386**

What this shows

KAVACH catches 920 more attack windows than the baseline while raising 1,229 fewer false alarms. Giving the baseline the same history does not help it (F1 falls to 0.46), so the gain comes from learning how traffic evolves, not from seeing more data.

5. Forecasting and World-Model Dynamics

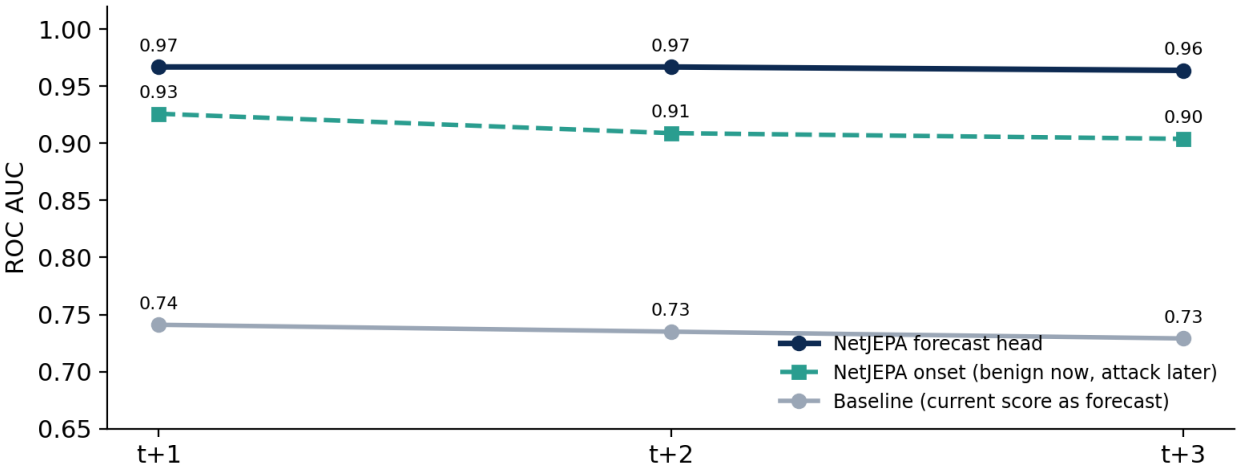


Figure 5.1 Forecast AUC for 1–3 windows ahead

Horizon	NetJEPA forecast head	NetJEPA current score	Onset forecast (benign now)	Baseline
t+1 (5 s)	0.967	0.957	0.926 (199 onsets)	0.741
t+2 (10 s)	0.967	0.952	0.909 (226 onsets)	0.735
t+3 (15 s)	0.964	0.947	0.904 (233 onsets)	0.729

5.1 Does temporal context help?

Configuration (same training budget)	F1	AUC	FPR	Recall
NetJEPA, context = 1 window	0.735	0.917	6.9%	0.661
NetJEPA, context = 3 windows	0.772	0.924	10.1%	0.755
NetJEPA v2 (full training)	**0.864**	**0.960**	**6.3%**	**0.856**

5.2 Transition model P(S_t+1 | S_t)

- **Gaussian negative log-likelihood:** 0.023 per latent dimension.
- **90% prediction-interval coverage:** 94.8%. The predicted uncertainty bands contain the true next state slightly more often than promised, so they are reliable (and slightly conservative).
- **Stage forecast:** macro-F1 0.74 / 0.72 / 0.70 and accuracy 92% / 92% / 91% at t+1 / t+2 / t+3.

6. Flow-Level and Packet-Level Features

The same model was scored on the 2,378 raw-PCAP validation windows with one traffic level hidden at inference, to measure what each contributes.

Input	F1	Precision	Recall	FPR	AUC
Flow + packet (both levels)	**0.872**	**0.817**	**0.935**	**0.6%**	**0.999**
Flow level only	0.624	0.723	0.548	0.6%	0.985
Packet level only	0.609	0.495	0.790	2.2%	0.954

Detection by source	NetJEPA F1 / AUC	Baseline F1 / AUC
Flow CSV windows (n = 5,665)	0.86 / 0.93	0.49 / 0.55
PCAP windows (n = 2,378)	0.87 / 1.00	0.50 / —

7. MITRE ATT&CK Stage Classification

Each window is mapped to one of six stages. DoS/DDoS windows are excluded here because their MITRE tactic (Impact) is not one of the six stages.

Stage	F1	Support	Comment
Benign	0.95	5,360	
Reconnaissance	0.91	265	Port scans reliably recognised
Initial Access	0.86	809	Web attacks and brute force
Command and Control	0.28	110	Botnet beaconing is often confused with benign traffic
Lateral Movement	—	0	No examples in the public validation data
Exfiltration	—	0	No examples in the public validation data

Overall: macro-F1 0.75 over the stages present, accuracy 92.4%.

7.1 Calibration of confidence

- **Expected calibration error:** 0.049 before and **0.033** after temperature scaling (T = 0.751).
- When KAVACH reports a given confidence, the observed attack frequency closely matches it (reliability diagram in Appendix A, Figure A.1).

8. Generalisation to Unseen Attack Families

Leave-one-attack-out test: the model is retrained from scratch with every window of one attack family removed, then asked to detect that family. This is the hardest test in the report and the one the problem statement asks about directly.

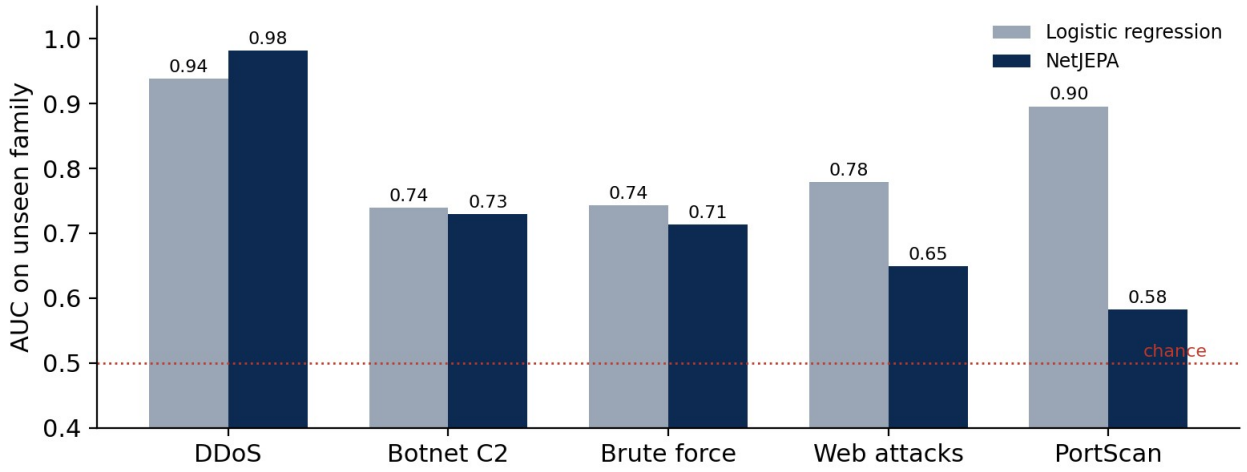


Figure 8.1 AUC on attack families never seen in training

Held-out family	NetJEPA AUC	NetJEPA recall	NetJEPA FPR	Logistic regression AUC / FPR
DDoS	**0.98**	**95%**	7.2%	0.94 / 33.9%
Botnet C2	0.73	13%	6.1%	0.74 / 30.1%
Brute force (FTP/SSH)	0.71	1%	1.0%	0.74 / 19.3%
Web attacks	0.65	2%	10.2%	0.78 / 28.5%
PortScan	**0.58**	**5%**	8.4%	0.90 / 26.7%

Honest finding

KAVACH transfers strongly to families related to what it has seen (unseen DDoS: AUC 0.98, 95% caught). It is weak on families unlike anything in training, especially PortScan. The baseline scores a higher AUC on some held-out families, but only by flagging 19–34% of benign traffic. A latent-space novelty score was also tested and did not close the gap, so it is not deployed. **Planned mitigation:** a rule-based layer for scan/flood/brute-force patterns, per-host baselines, and training on CSE-CIC-IDS-2018, CTU-13 and site traffic.

9. Explainability Validation

KAVACH explains predictions at six levels. Each was tested quantitatively rather than only displayed.

9.1 Linear probes: what each layer knows

Concept (balanced accuracy)	Input	HGT 1	HGT 2	GRU	Context	Predictor
Is scanning	0.80	0.92	0.93	0.93	0.93	0.92
MITRE stage (5 classes)	0.51	0.72	0.72	0.75	0.75	0.81
Is escalating	0.63	0.72	0.71	0.75	0.75	0.78
Attack within 3 windows	0.57	0.82	0.84	0.86	0.86	0.90

Kill-chain stage and escalation become decodable only after the temporal (GRU) layer, which is what the architecture is designed to do. Chance level is 0.5.

9.2 Tuned lens: where the decision forms

Layer	Input	HGT 1	HGT 2	Spatial	GRU	Context
Infiltration AUC	0.77	0.90	0.85	0.90	0.96	0.96

9.3 Other interpretability checks

Method	Result
Sparse autoencoder	4,096-feature dictionary; 1,231 active features, 641 labelled, 88 attack-specific; 0.24% variance unexplained
Causal tracing	Patching the graph encoder closes 84% (Reconnaissance), 78% (C2) and 88% (Initial Access) of the attack-vs-benign gap: the decision is made in the graph layers
Explanation consistency	Mean Jaccard 0.57 between top-5 Integrated-Gradients features of same-type attacks (0.48–0.65 per family): moderately stable reasoning
Concept bottleneck	10 human-readable concepts predicted with AUC 0.96–1.00; stage macro-F1 0.41 through the transparent pathway (analyst can override any concept)
Representation health	Embedding std 0.71 (healthy, > 0.5): no representation collapse

10. System-Level Tests

10.1 Packet-parser throughput

The custom parser was timed against the Scapy-based path on the first 20,000 packets of the demo capture, on the same 2-vCPU machine.

```
Capture: KAVACH_demo_100hosts_DoS.pcap, first 20,000 packets, 2 vCPU, Python 3.11.15
scapy path : 5.54s  (3,608 packets/s)
fastpcap   : 0.095s (210,492 packets/s)
speedup    : 58.3x
```

A unit test (test_fastpcap_matches_scapy_fields) confirms both parsers produce identical fields.

10.2 Ledger integrity

```
$ python -m src.cli verify-chain
Chain intact: 15 blocks verified from genesis to tip, no tampering detected.
Training data on disk matches the manifest chained at block 4.
```

- **Tamper detection:** unit tests modify and delete ledger blocks and confirm verification fails at the exact block (test_modification_is_detected, test_deletion_is_detected).
- **Smart contracts:** IP blocking needs three independent nodes; unacknowledged attacks are escalated; poisoned analyst feedback is flagged (test_contracts_and_audit.py).
- **Training-data integrity:** the training files on disk match the manifest chained at block 4.

10.3 Three-tier access control

Test	Verified behaviour
Factor count per tier	Routine: password only · Sensitive: + authenticator (TOTP) · Mission-critical: + hardware-key challenge
TOTP correctness	Matches the RFC 6238 reference test vector
Lockout	5 failed attempts lock the account; every failure and the lockout are chained on the ledger
Response rules	Routine auto-executes · Sensitive requires confirmation (rejected otherwise) · Mission-critical queues for approval
Two-person rule	Requester cannot approve own action; non-cleared users cannot approve; a second officer can; ledger stays valid

10.4 End-to-end run on a real attack capture

A 2-minute slice of the CIC-IDS-2017 Wednesday capture (100 hosts, onset of the DoS Hulk attack) was analysed fully offline with the command-line tool:

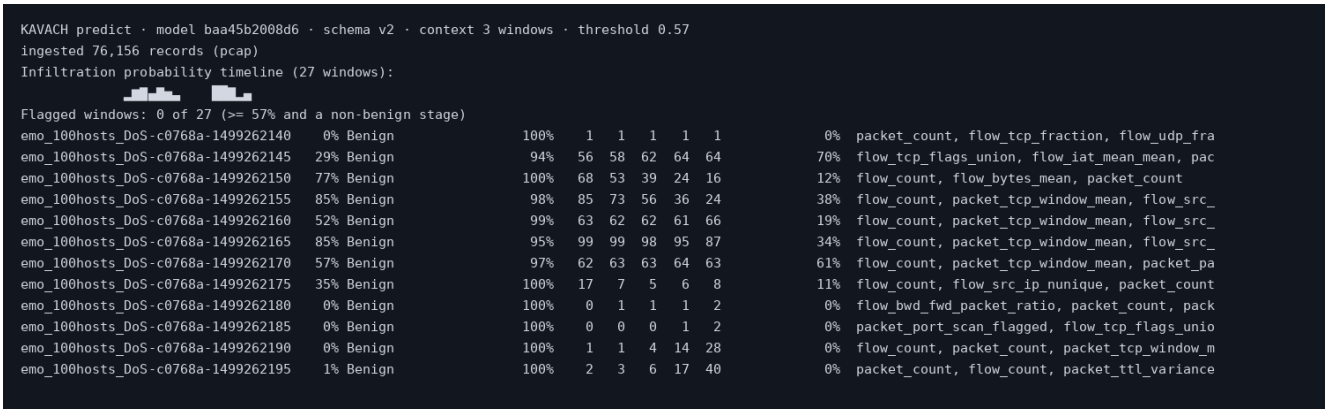


Figure 10.1 CLI output: risk rises from 0% to 77–100% as the attack begins, with forecasts rising a window earlier

Risk stays at 0–5% during benign traffic, the forecast rises to 56–64% one window before the flood, and risk reaches 99–100% at the peak. The stage column reads "Benign" because DoS belongs to the MITRE Impact tactic, which is outside the six labelled stages.

11. Robustness and Known Limitations

Area	Finding
Adversarial robustness	PGD attack in feature space: flipping a detection needed on average only 0.06 σ, touching about 1 feature. The current checkpoint is not adversarially robust. In real traffic these features are tied to the attack itself (scanning fewer ports means less reconnaissance), so the bound favours the attacker.
Unseen families	Weak on attack families unlike the training data (PortScan AUC 0.58). See Section 8.
Rare stages	Lateral movement and exfiltration have no examples in the public validation data, so they are untested.
Cross-dataset	Parsers for CSE-CIC-IDS-2018 and CTU-13 are implemented and unit-tested; those datasets were not yet evaluated.
Lab data	All results come from testbed traffic; each deployment site needs a calibration phase.

> **Mitigation plan**
>
> Noise and adversarial augmentation during training; a transparent rule layer and per-host baselines alongside NetJEPA so an attacker must evade several independent detectors; cross-dataset training; and per-site recalibration of normalisation, temperature and threshold.

Appendix A: Screenshots of the Live Benchmark Page

The Benchmark page recomputes every figure from the served checkpoint and is cached per model hash. Screenshots taken 29 September 2026.

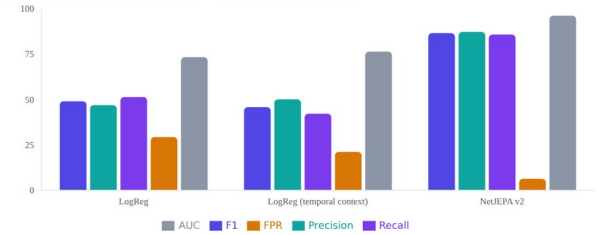
L1

Output — do the outputs match reality?

Detection, calibration, forecasting and stage mapping vs a context-free baseline.

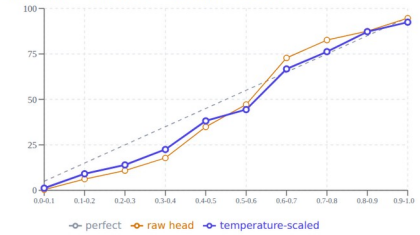
NETJEPA F1	PRECISION	RECALL	FALSE POSITIVE RATE	ROC AUC
0.86	**0.87**	**0.86**	**6.3%**	**0.96**
+76.5% vs baseline (0.49)	2297 TP / 339 FP	386 attacks missed	baseline 29.3%	baseline 0.73

Binary infiltration detection (%) — same held-out windows



8,043 validation windows (last 20% of every CIC-IDS-2017 day, never trained on). Decision threshold 0.57 on the calibrated probability, chosen on a separate calibration split.

Calibration — ECE 0.049 → 0.033 (T = 0.751)



Observed attack frequency per predicted-probability bin. On the diagonal, "82%" really means ~82%.

Forecasting: AUC for 'attack present k windows later'



t+0 = detection on the current window; t+k = forecast head on the predictor's k-step rollout vs the label of window t+k.

MITRE stage head — macro-F1 0.75, accuracy 92.4%

true \ predicted	B	R	IA	LM	CaC	E	F1
Benign (5360)	5009	4	216	26	105	0	0.95
Reconnaissance (265)	39	225	0	1	0	0	0.91
Initial Access (809)	30	0	777	0	2	0	0.86
Lateral Movement (0)	0	0	0	0	0	0	–
Command and Control (110)	65	0	10	0	35	0	0.28
Exfiltration (0)	0	0	0	0	0	0	–

Macro-F1 over stages present in the validation split. DoS/DDoS windows are excluded (MITRE 'Impact' is not one of the 6 classes).

Figure A.1 Output level: detection, calibration, forecasting and stage mapping

L1

World-model dynamics — temporal context & forward simulation

Is this more than a static classifier? History, K-step forecasts and the probabilistic transition model.

Does temporal context help? (NetJEPA sees 3 windows)

model	F1	AUC	FPR
Logistic Regression (baseline)	0.49	0.73	29.3%
Logistic Regression (temporal context)	0.46	0.76	21.1%
NetJEPA v2	**0.86**	**0.96**	**6.3%**
NetJEPA retrained, context = 1 (4 ep.)	0.74	0.92	6.9%
NetJEPA retrained, context = 3 (4 ep.)	0.77	0.92	10.1%

The temporal logistic regression sees the same 3 windows NetJEPA does, so any gap is the value of learned dynamics, not of extra input.

Forward simulation: predicting attack k windows ahead

horizon	forecast head AUC	current score AUC	onset AUC (benign now)
t+1	**0.97**	0.96	0.93 (199)
t+2	**0.97**	0.95	0.91 (226)
t+3	**0.96**	0.95	0.90 (233)

Transition model $p(S_{t+1}|S_t)$: Gaussian NLL 0.023 per dim; the predicted 90% interval covers the true next state 95% of the time.

"Onset" restricts to windows that are benign now: can the model see an attack coming before it starts?

Figure A.2 World-model dynamics: temporal context and forward simulation

L1

Both traffic levels & unseen attacks

What each traffic level contributes, and detection of attack families never seen in training.

Flow level + packet level (joint PCAP windows)

input	F1	AUC	recall	FPR
flow + packet (both levels)	**0.87**	**1.00**	**0.94**	**0.6%**
flow level only	0.62	0.98	0.55	0.6%
packet level only	0.61	0.95	0.79	2.2%

Flow CSV (flow level): F1 0.86 · AUC 0.93 · n=5665
PCAP (both levels): F1 0.87 · AUC 1.00 · n=2378

Same model, same validation windows, with one traffic level hidden at inference -- measures what each level contributes.

Generalisation to unseen attack families (leave-one-out retraining)

held-out family	NetJEPA AUC	recall	FPR	LR AUC	LR recall	LR FPR
Botnet C2 (Command and Control) (463)	**0.73**	13%	6%	0.74	61%	30%
Brute force (FTP/SSH-Patator) (2345)	**0.71**	1%	1%	0.74	41%	19%
DDoS (1303)	**0.98**	95%	7%	0.94	91%	34%
PortScan (Reconnaissance) (1721)	**0.58**	5%	8%	0.90	94%	27%
Web attacks (Initial Access) (461)	**0.65**	2%	10%	0.78	67%	29%

Each row retrains from scratch with every window of that family removed from training; scores are the family's windows vs benign validation windows. Neither model has ever seen the family. Finding: NetJEPA transfers to DDoS (it has seen the DoS family) but at its deployed threshold it misses most windows of families unlike anything in training (PortScan, brute force, web, botnet); the baseline catches more of them only by flagging 19-30% of benign windows. Latent-space novelty scoring did not close this gap (reported in the JSON as netjepa_novelty).

Figure A.3 Both traffic levels and leave-one-attack-out generalisation

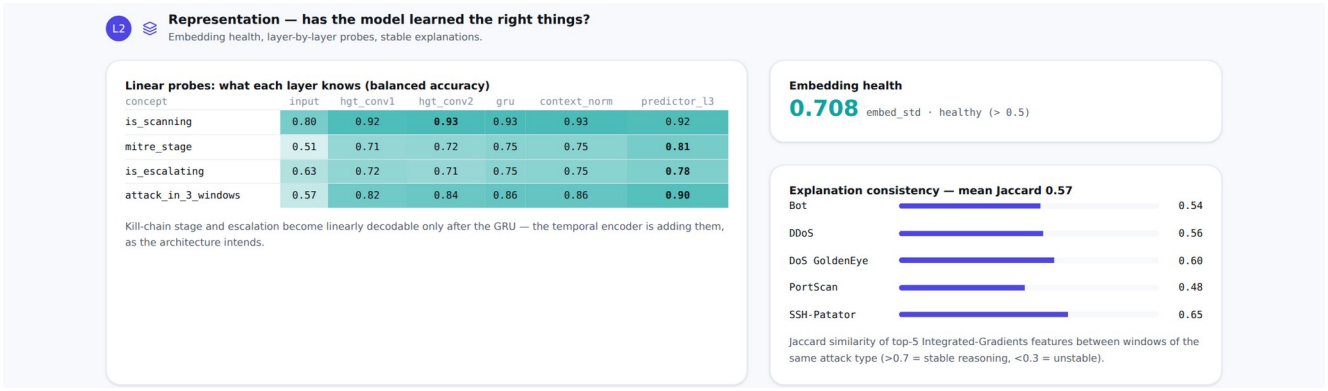


Figure A.4 Representation: linear probes, embedding health, explanation consistency

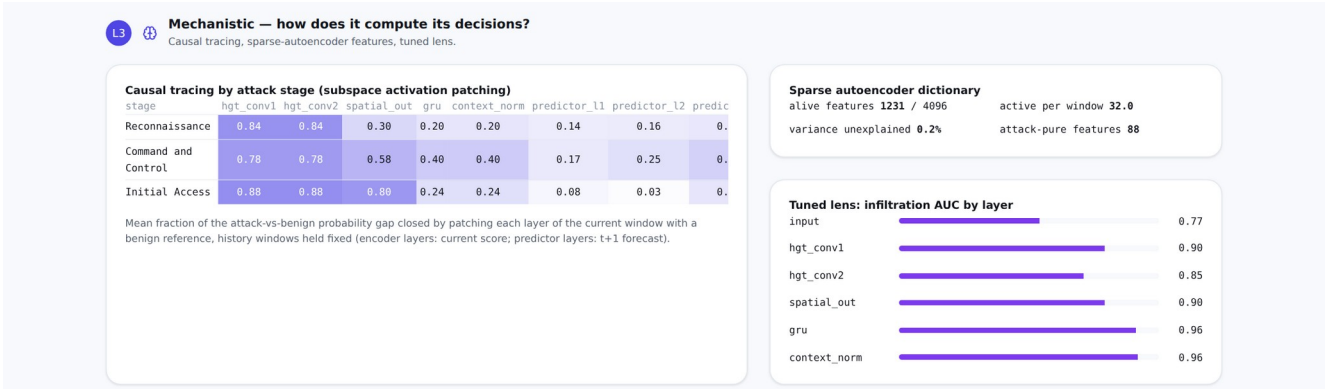


Figure A.5 Mechanism: causal tracing, sparse autoencoder, tuned lens

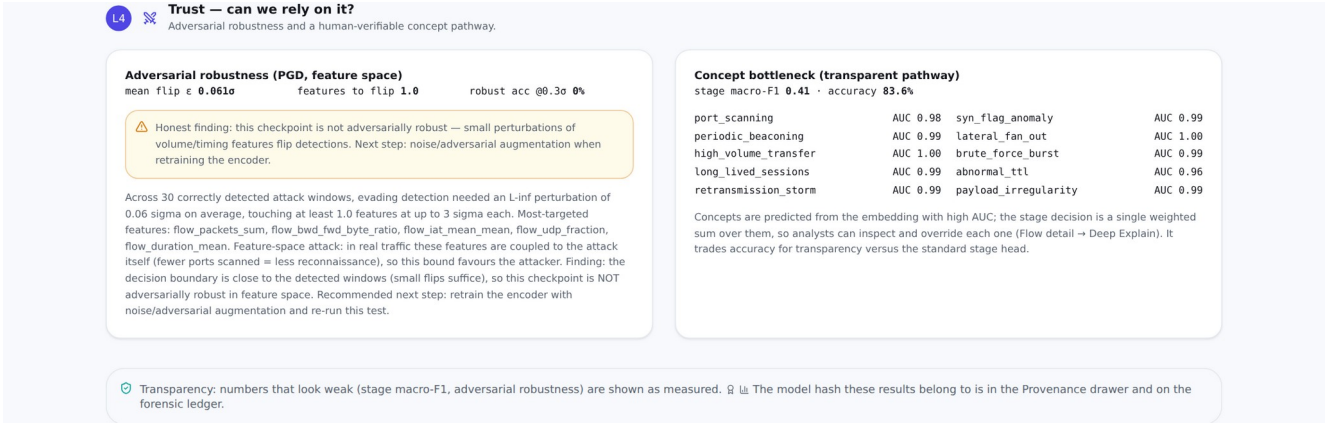


Figure A.6 Trust: adversarial robustness and concept bottleneck

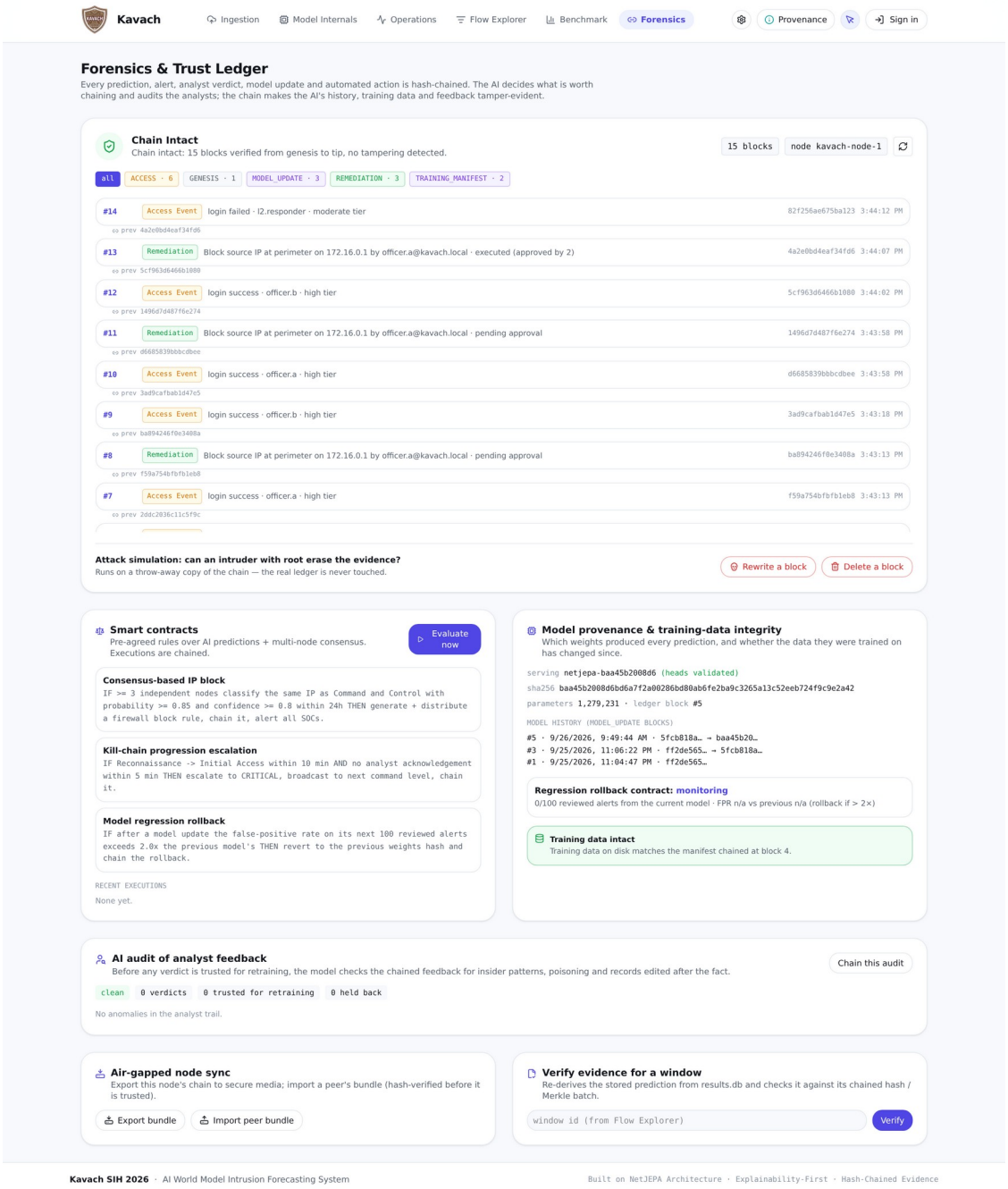


Figure A.7 Forensics page: ledger status, smart contracts, provenance and verification

Appendix B: Artefacts and Reproduction

Artefact	Location / fingerprint
Model weights	`data/processed/netjepa_weights.pt · SHA-256 baa45b20…c9e2a42`
Benchmark results (JSON)	`data/processed/benchmark_cache.json · SHA-256 adcb6192… 4b24479755`
Training report	`data/processed/v2_report.json`
Generalisation runs	`data/processed/generalization/loao_*.json, context_*.json`
Interpretability artefacts	`data/processed/interpretability/`
Demo capture	`data/raw/uploads/KAVACH_demo_100hosts_DoS.pcap`

Commands to reproduce

```
# 1. run the full test suite
python -m pytest -v
```

```
# 2. recompute the benchmark from the served checkpoint
python -m src.cli benchmark --refresh

# 3. model identity and ledger integrity
python -m src.cli info
python -m src.cli verify-chain

# 4. analyse a capture offline
python -m src.cli predict data/raw/uploads/KAVACH_demo_100hosts_DoS.pcap --all
```

Appendix C: Full Test Log

Output of `python -m pytest -v`, 29 September 2026.

```
============================= test session starts ==============================
platform linux -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
rootdir: /home/claude/kv2
plugins: anyio-4.13.0
tests/test_adapters.py::test_row_to_flow_maps_real_fields PASSED
tests/test_adapters.py::test_row_to_flow_uses_real_host_ips_when_present PASSED
tests/test_adapters.py::test_risk_level_thresholds PASSED
tests/test_adapters.py::test_row_to_alert_maps_fields PASSED
tests/test_adapters.py::test_kill_chain_state_marks_latest_stage_active PASSED
tests/test_adapters.py::test_kill_chain_state_empty_store PASSED
tests/test_adapters.py::test_risk_score_response_with_data PASSED
tests/test_adapters.py::test_risk_score_response_no_data PASSED
tests/test_adapters.py::test_risk_score_response_flags_untrained_model PASSED
tests/test_api_extended.py::test_benchmark_without_windows_cache_is_structured PASSED
tests/test_api_extended.py::test_feedback_is_persisted_and_chained PASSED
tests/test_api_extended.py::test_risk_trend_timeline_filters_and_audit PASSED
tests/test_api_extended.py::test_login_remediation_and_tamper_demo PASSED
tests/test_auth_tiers.py::test_primitives PASSED
tests/test_auth_tiers.py::test_factor_count_scales_with_tier PASSED
tests/test_auth_tiers.py::test_wrong_password_lockout_and_ledger PASSED
tests/test_auth_tiers.py::test_me_and_logout PASSED
tests/test_auth_tiers.py::test_response_rules_per_tier PASSED
tests/test_auth_tiers.py::test_policy_and_demo_authenticator PASSED
tests/test_benchmark.py::test_evaluate_baseline_learns_a_separable_signal PASSED
tests/test_benchmark.py::test_run_benchmark_reports_untrained_netjepa_when_no_checkpoint PASSED
tests/test_benchmark.py::test_run_benchmark_evaluates_netjepa_when_checkpoint_present PASSED
tests/test_benchmark.py::test_run_benchmark_caches_until_checkpoint_changes PASSED
tests/test_blockchain_ledger.py::test_genesis_and_valid_chain PASSED
tests/test_blockchain_ledger.py::test_modification_is_detected PASSED
tests/test_blockchain_ledger.py::test_deletion_is_detected PASSED
tests/test_blockchain_ledger.py::test_simulated_tamper_leaves_real_chain_intact PASSED
tests/test_blockchain_ledger.py::test_low_risk_windows_are_merkle_batched PASSED
tests/test_blockchain_ledger.py::test_find_blocks_for_window_includes_batches PASSED
tests/test_blockchain_ledger.py::test_sync_bundle_roundtrip_and_fork_detection PASSED
tests/test_blockchain_ledger.py::test_model_update_only_recorded_on_change PASSED
tests/test_contracts_and_audit.py::test_consensus_needs_three_independent_nodes PASSED
tests/test_contracts_and_audit.py::test_escalation_when_unacknowledged PASSED
tests/test_contracts_and_audit.py::test_feedback_audit_flags_subnet_shielding_and_db_edits PASSED
tests/test_dataset_formats.py::test_detect_formats PASSED
tests/test_dataset_formats.py::test_cicids2018_uses_real_timestamps_protocol_and_flags PASSED
tests/test_dataset_formats.py::test_cicids2017_full_keeps_ips PASSED
tests/test_dataset_formats.py::test_ctu13_binetflow PASSED
tests/test_extract.py::test_window_flow_only_buckets_by_window_seconds PASSED
tests/test_extract.py::test_window_packet_only_buckets_and_aggregates PASSED
tests/test_extract.py::test_window_packet_records_capture_real_edges PASSED
tests/test_extract.py::test_build_feature_windows_accepts_dataframe_packet_records PASSED
tests/test_extract.py::test_normalize_zero_means_unit_std_per_column PASSED
tests/test_extract.py::test_rows_per_window_ignores_synthetic_timestamp_spread PASSED
tests/test_extract.py::test_empty_inputs_return_empty_dataframe PASSED
tests/test_extract.py::test_iter_feature_windows_yields_dicts_in_order PASSED
tests/test_extract.py::test_merge_flow_and_packet_windows_on_window_start PASSED
tests/test_ingestion.py::test_load_flow_csv_normalizes_schema PASSED
tests/test_ingestion.py::test_load_flow_csv_anchors_to_real_pcap_start PASSED
tests/test_ingestion.py::test_load_flow_csv_falls_back_to_guess_when_pcap_missing PASSED
tests/test_ingestion.py::test_load_flow_dir_matches_csvs_to_pcaps_by_name PASSED
tests/test_ingestion.py::test_iter_flow_csv_batches_sum_to_full_file PASSED
tests/test_ingestion.py::test_parse_pcap_extracts_features_and_retransmission PASSED
tests/test_ingestion.py::test_parse_pcap_flags_port_scan PASSED
tests/test_ingestion.py::test_iter_packets_batches PASSED
tests/test_netjepa.py::test_forward_batch_network_only_shapes PASSED
tests/test_netjepa.py::test_forward_batch_with_hosts PASSED
tests/test_netjepa.py::test_forward_with_context_and_target_batches PASSED
tests/test_netjepa.py::test_batchnorm_keeps_embeddings_spread_out_not_collapsed PASSED
tests/test_netjepa.py::test_rollout_produces_k_steps_of_the_right_shape PASSED
tests/test_netjepa.py::test_update_target_encoder_moves_target_params_toward_context PASSED
tests/test_netjepa.py::test_vicreg_loss_runs_on_model_output PASSED
```

```
tests/test_netjepa.py::test_eval_mode_supports_single_example_inference PASSED
tests/test_pipeline.py::test_run_pipeline_placeholder_when_no_capture PASSED
tests/test_pipeline.py::test_run_pipeline_streams_real_csv_ingestion PASSED
tests/test_pipeline.py::test_run_pipeline_caps_windows_processed_through_model_stages PASSED
tests/test_pipeline.py::test_run_pipeline_streams_real_feature_extraction_from_csv PASSED
tests/test_pipeline.py::test_run_pipeline_feature_extraction_placeholder_when_no_capture PASSED
tests/test_pipeline.py::test_run_pipeline_model_stages_run_untrained_by_default PASSED
tests/test_pipeline.py::test_run_pipeline_model_stages_use_checkpoint_when_present PASSED
tests/test_pipeline.py::test_run_pipeline_streams_real_pcap_ingestion PASSED
tests/test_results_store.py::test_save_and_recent_predictions_roundtrip PASSED
tests/test_results_store.py::test_recent_predictions_respects_limit PASSED
tests/test_results_store.py::test_get_prediction_by_window_id PASSED
tests/test_results_store.py::test_alerts_filters_by_probability_threshold PASSED
tests/test_results_store.py::test_summary_stats_aggregates_correctly PASSED
tests/test_results_store.py::test_summary_stats_empty_store PASSED
tests/test_results_store.py::test_clear_removes_all_rows PASSED
tests/test_scoring.py::test_infiltration_head_single_vector_in_unit_range PASSED
tests/test_scoring.py::test_infiltration_head_batch PASSED
tests/test_scoring.py::test_score_infiltration_curve_length_matches_rollout PASSED
tests/test_scoring.py::test_score_infiltration_empty_rollout PASSED
tests/test_scoring.py::test_attack_stage_classifier_output_shape PASSED
tests/test_scoring.py::test_attack_stage_predict_returns_valid_distribution PASSED
tests/test_scoring.py::test_map_attack_stage_convenience_function PASSED
tests/test_scoring_extras.py::test_temperature_scaling_reduces_overconfidence PASSED
tests/test_scoring_extras.py::test_remediation_uses_real_ips_and_covers_every_stage PASSED
tests/test_scoring_extras.py::test_concept_bottleneck_shapes_and_overrides PASSED
tests/test_shap_explainer.py::test_explain_infiltration_score_returns_real_feature_names PASSED
tests/test_shap_explainer.py::test_explain_attack_stage_for_a_specific_class PASSED
tests/test_shap_explainer.py::test_explain_prediction_respects_n_features_cap PASSED
tests/test_shap_explainer.py::test_explain_prediction_with_custom_feature_names PASSED
tests/test_state_builder.py::test_network_only_window_has_no_host_nodes PASSED
tests/test_state_builder.py::test_packet_edges_become_host_nodes_and_flow_edges PASSED
tests/test_state_builder.py::test_hetero_data_shapes_match_hosts_and_canonical_feature_dim PASSED
tests/test_state_builder.py::test_hetero_data_falls_back_gracefully_with_no_hosts PASSED
tests/test_state_builder.py::test_two_different_window_shapes_still_produce_same_width_vector PASSED
tests/test_train.py::test_infiltration_pos_weight_balanced_dataset PASSED
tests/test_train.py::test_infiltration_pos_weight_imbalanced_dataset PASSED
tests/test_train.py::test_infiltration_pos_weight_no_positives_falls_back_to_one PASSED
tests/test_v2_components.py::test_fastpcap_matches_scapy_fields PASSED
tests/test_v2_components.py::test_joint_windows_have_both_levels_and_timeline_labels PASSED
tests/test_v2_components.py::test_attack_timeline_labels_only_attacker_victim_in_interval PASSED
tests/test_v2_components.py::test_feature_schema_v2_has_both_levels_and_modality_flags PASSED
tests/test_v2_components.py::test_batched_encoder_matches_single_path_and_predictor_is_gaussian PASSED
tests/test_v2_components.py::test_examples_carry_future_labels_for_transition_supervision PASSED
tests/test_v2_components.py::test_cli_predict_runs_offline PASSED
tests/test_v2_components.py::test_novelty_detector_percentiles PASSED
tests/test_vicreg.py::test_vicreg_loss_runs PASSED
tests/test_vicreg.py::test_vicreg_penalizes_collapse PASSED

====================== 110 passed, 1 warning in 41.40s =======================
```